

TCP and QUIC Performance under Packet Loss on Low- and High-Latency FABRIC Paths

Manh Cuong La

Department of Electrical and Computer Engineering

University of Massachusetts Amherst

Amherst, MA, USA

mla@umass.edu

Abstract—Quick UDP Internet Connections (QUIC) is a modern transport protocol designed to improve web transport over the User Datagram Protocol. The Transmission Control Protocol (TCP) remains one of the dominant reliable transport protocols on the Internet. This project compares TCP and QUIC on the FABRIC testbed under controlled packet loss across two real network paths: MASS to NEWY and MASS to UCSD. The MASS to NEWY path had a round trip time of about 4.67 ms, while the MASS to UCSD path had a round trip time of about 71.57 ms. Each path was tested under zero percent, two percent, and five percent packet loss with a fixed eight megabyte payload. The main finding is that path latency dramatically increases the cost of packet loss. At two percent loss, TCP degraded by only 2.84 percent on the low-latency path but by 97.46 percent on the high-latency path. QUIC showed the same directional pattern. The bandwidth-delay product explains this difference because higher latency places more data in flight per lost packet, which makes each loss event more expensive to recover from. TCP retransmission counts confirmed that injected loss directly affected transport recovery behavior.

Index Terms—QUIC, TCP, FABRIC, packet loss, throughput, latency

I. INTRODUCTION

Transport protocols are important because they control how data is delivered across the Internet. The Transmission Control Protocol (TCP) has been one of the most widely used transport protocols for many years. It provides reliable and ordered byte stream delivery, which makes it suitable for many applications. However, TCP can lose performance when packets are dropped. Packet loss can trigger retransmissions and congestion control responses. These responses may reduce the sending rate.

QUIC is a younger transport protocol that runs over the User Datagram Protocol (UDP). It provides reliability, stream support, congestion control, and security inside the protocol [1]. QUIC is also the transport protocol used by Hypertext Transfer Protocol Version 3 (HTTP/3) [2]. This makes QUIC important for modern web applications. In theory, QUIC can reduce connection setup latency and improve stream handling. However, better protocol design does not always lead to better measured performance in every network condition.

This project focuses on how TCP and QUIC behave under packet loss on real network paths. Many transport studies use simulation or local emulation. Those methods are useful because they allow careful control. However, real wide-area

paths can behave differently. Real paths include actual routing, site resources, queue behavior, and shared infrastructure. For this reason, FABRIC was used in this project to create real testbed paths across different sites in the United States.

Two paths were chosen for this experiment. The first path used MASS to NEWY and had a low round-trip time (RTT). The second path used MASS to UCSD and had a much higher RTT. By comparing these two paths, the experiment studied whether packet loss affects low-latency and high-latency paths differently. The measured metrics were throughput, RTT, and TCP retransmissions.

This project is also useful because it separates protocol design from measured performance. QUIC includes features such as stream multiplexing, encryption, and faster connection setup. However, a protocol with useful design features can still show lower throughput if the implementation, path, or test condition is different. Because of this, the experiment does not only ask which protocol is faster. It also asks how each protocol changes when packet loss and path latency change.

The project also has a practical motivation. Network users often care about how quickly a fixed amount of data can be delivered. This can include file transfers, media chunks, web objects, and application data. For that reason, this experiment used a fixed 8 MB transfer size instead of a long continuous throughput test. This made the TCP and QUIC workloads comparable across paths and loss levels.

II. RELATED WORK

TCP is defined as a reliable transport protocol for the Internet Protocol stack [3]. TCP uses sequence numbers, acknowledgments, retransmissions, and congestion control. These features allow TCP to recover from lost data and adjust to network congestion. When TCP detects packet loss, it may retransmit data and reduce its congestion window. This protects the network, but it can also reduce throughput.

QUIC also provides reliable transport, but it runs on top of UDP instead of directly over IP like TCP [1]. QUIC includes loss recovery and congestion control inside the protocol implementation. It also supports multiple streams inside one connection. This can reduce some stream-level blocking problems found in TCP-based web transport. HTTP/3 uses QUIC as its transport layer, which makes QUIC important for current web systems [2].

Several prior studies have compared QUIC and TCP. Nepomuceno et al. compared QUIC and TCP under different RTT, packet loss, web page, and cache settings [4]. Their study showed that QUIC did not always outperform TCP. They also found that RTT had a strong effect on page load time. This supports the decision to test two paths with very different latency.

Yu et al. studied QUIC and TCP from a transport-layer perspective [5]. Their results showed that QUIC can be less competitive than TCP in networks with low loss, large buffers, or large propagation delay. They also found that QUIC can perform better in some lossy or constrained network conditions. This is related to this project because the experiment evaluates QUIC and TCP under different loss and latency conditions.

Kakhki et al. argued that rigorous QUIC evaluation is difficult because QUIC evolves quickly and different implementations can behave very differently [6]. This point is important for this experiment. TCP was tested with iperf3, which uses the optimized Linux TCP stack. QUIC was tested with a custom Python aioquic program. Because the tools are not equally optimized, the absolute throughput numbers should be interpreted carefully. The main focus is the degradation trend across packet loss levels and path latency.

Overall, prior work shows that QUIC should not be treated as better than TCP in every condition by default. Performance depends on network conditions such as RTT, loss rate, buffer behavior, and implementation details [4]–[6]. This project follows that idea by comparing TCP and QUIC on two FABRIC paths with very different latency.

FABRIC is a national-scale programmable research testbed [7]. It allows users to create slices across multiple sites. This makes FABRIC useful for experiments that need real network paths rather than only local emulation. In this project, FABRIC allowed comparison between a short regional path and a higher-latency long-distance path under the same experiment structure.

This project differs from prior work in two main ways. First, it uses FABRIC slices to test real wide-area paths instead of relying only on local emulation. Second, it uses a simple fixed-payload experiment, which makes the comparison repeatable across paths, protocols, and loss levels.

III. METHOD

This section describes how the FABRIC experiment was organized and how the TCP and QUIC tests were run. The goal was to keep the two protocol tests as similar as possible. Both protocols used the same paths, same packet loss levels, same number of trials, and same 8 MB transfer size.

A. Experiment Overview

The experiment compared TCP and QUIC on two FABRIC paths. The first path was MASS to NEWY, which represented a low-latency regional path. The second path was MASS to UCSD, which represented a higher-latency long-distance path. Each path was tested under three packet loss levels: 0%, 2%, and 5%. For each loss level, TCP and QUIC were each tested

TABLE I
EXPERIMENT WORKFLOW

Step	Description
1	Create two FABRIC slices for MASS to NEWY and MASS to UCSD
2	Configure client and server data interfaces with manual IPv4 addresses
3	Install required tools on each node
4	Start TCP and QUIC servers on the server nodes
5	Apply packet loss on the client outbound interface
6	Measure latency with ping
7	Run TCP and QUIC transfer tests
8	Save raw results, summary results, and graphs

three times. This created 36 results in total. Table I summarizes the experiment workflow.

B. FABRIC Slice Setup

This experiment used two separate FABRIC slices. Each slice had one client node and one server node. The MASS to NEWY slice tested a short regional path. The MASS to UCSD slice tested a longer path with much higher RTT.

Both slices used Ubuntu 22.04 nodes. Each node used 2 CPU cores, 4 GB of RAM, and a 10 GB disk to stay within the resource settings available for this FABRIC project. Each node also had a basic data network interface.

The client and server in each slice were connected using a FABRIC Layer 2 site-to-site (L2STS) network. This created a layer 2 site-to-site connection between the two FABRIC sites. After the slices became active, Internet Protocol version 4 (IPv4) addresses were manually assigned to the data interfaces. The MASS to NEWY path used 10.10.1.1 for the client and 10.10.1.2 for the server. The MASS to UCSD path used 10.10.2.1 for the client and 10.10.2.2 for the server.

The notebook submitted the two slices in parallel. This reduced setup time and kept the two paths in the same experiment run. After each slice became stable, the notebook waited for Secure Shell (SSH) access and refreshed the slice handle. This made sure that later commands ran on active nodes.

C. Software Tools

TCP was tested with iperf3. The TCP server ran on port 5201. The TCP client used JavaScript Object Notation (JSON) output so that throughput and retransmission values could be parsed by the notebook. The TCP test used a fixed-size transfer with the `-n 8M` option. This matched the 8 MB transfer size used by the QUIC script.

QUIC was tested with a custom Python script based on aioquic. The QUIC server ran on port 4433. A temporary self-signed certificate was created on each server node because QUIC requires Transport Layer Security (TLS)-based security. The QUIC client opened a QUIC stream, sent data to the server, and printed a JSON result.

The TCP and QUIC results were stored in the same format. Each row included the path name, protocol, loss level, trial number, throughput, latency, and timestamp. TCP rows also included retransmission counts from iperf3.

D. QUIC Test Implementation

The QUIC script used a simple client and server design. The client sent a short JSON header that told the server how many bytes to expect. The client then sent the payload in 16 KB chunks. The server counted the number of bytes received. After the expected amount of data arrived, the server returned a JSON result with received bytes, duration, and throughput.

This design made the QUIC output easy to parse. It also made the QUIC result format similar to the TCP result format. However, it did not expose internal QUIC recovery information. The script measured throughput and completed bytes, but it did not report QUIC packet loss events or QUIC retransmission counters.

Before the main experiment, the notebook ran a short QUIC validation test on both paths. The validation test successfully transferred 8 MB on MASS to NEWY and MASS to UCSD. This confirmed that the QUIC script, certificate setup, server process, and client command were working before the final tests.

E. Transfer Size

Both protocols were configured to send the same 8 MB payload in each trial. This made the TCP and QUIC workloads consistent. The fixed transfer size also helped the high-loss tests finish reliably.

This choice means the results should be interpreted as fixed-payload transfer results. They should not be treated as long-duration maximum throughput results. A longer test could produce different absolute throughput values because congestion control would have more time to stabilize.

F. Packet Loss Setup

Packet loss was applied on the client outbound data interface. Linux `tc netem` was used to control the loss rate. The experiment used three loss levels: 0%, 2%, and 5%.

At 0% loss, the notebook removed the netem rule and used the normal queueing behavior. At 2% and 5% loss, the notebook replaced the queueing discipline with the selected netem loss rule.

The notebook printed the qdisc setting before each group of trials. At 0% loss, the interface showed the normal queueing configuration. At 2% and 5% loss, the interface showed a netem rule with the selected loss percentage. This helped verify that the loss setting was applied before each test group.

Packet loss was applied only in the client-to-server direction. This means the loss affected the main data transfer direction for both TCP and QUIC. This design kept the loss model simple and repeatable.

G. Trial Design

For each path, the notebook ran the same test sequence. First, it applied the selected packet loss level. Next, it measured RTT using ping. Then, it ran the TCP transfer test. After that, it ran the QUIC transfer test. This process was repeated three times for each loss level.

Table II summarizes the trial matrix used in the experiment.

TABLE II
TRIAL MATRIX

Path	Loss Levels	Protocols	Trials
MASS to NEWY	0%, 2%, 5%	TCP, QUIC	3 each
MASS to UCSD	0%, 2%, 5%	TCP, QUIC	3 each

This produced 36 total measurements: 2 paths, 3 loss levels, 2 protocols, and 3 trials per condition.

H. Data Processing and Validation

After the tests finished, the notebook saved three comma-separated values (CSV) files. The raw results file stored every trial. The summary file averaged the three trials for each path, protocol, and loss level. The degradation summary compared each loss result against the 0% loss baseline for the same path and protocol.

The notebook also generated throughput graphs. These graphs plotted packet loss on the x-axis and average throughput on the y-axis. Separate graphs were created for MASS to NEWY and MASS to UCSD.

The notebook reported 36 total rows, 36 successful rows, and 0 error rows, which confirmed that all planned tests completed successfully. After the experiment, the cleanup function removed packet loss from the client interfaces and stopped the server processes.

IV. RESULTS

This section reports the final throughput, latency, degradation, retransmission, and trial variation results.

A. Path Latency

The two paths had very different RTT values. The MASS to NEWY path had an average RTT of about 4.67 ms. The MASS to UCSD path had an average RTT of about 71.57 ms. This confirms that the experiment compared a low-latency path with a high-latency path.

Table III summarizes the average RTT values for each path and loss level.

TABLE III
AVERAGE ROUND-TRIP TIME BY PATH AND LOSS LEVEL

Path	0% Loss RTT (ms)	2% Loss RTT (ms)	5% Loss RTT (ms)
MASS to NEWY	4.664	4.669	4.673
MASS to UCSD	71.565	71.584	71.576

Latency stayed almost constant across packet loss settings. This is expected because packet loss was applied as random outbound loss, not as extra delay. For this reason, the main results focus on throughput and degradation rather than latency graphs. The latency values were still important because they suggest that the two paths had very different delay conditions.

B. MASS to NEWY Throughput

Fig. 1 shows the throughput results on the MASS to NEWY path. At 0% loss, TCP reached 594.14 Mbps, while QUIC reached 119.72 Mbps. TCP had a higher baseline throughput because iperf3 uses the optimized Linux TCP stack. The QUIC test used a Python aioquic script, which has more user-space overhead.

At 2% loss, TCP stayed close to its baseline and reached 577.28 Mbps. This was only a 2.84% drop. QUIC dropped to 13.52 Mbps, which was an 88.71% drop from its baseline. At 5% loss, both protocols degraded dramatically. TCP dropped to 15.68 Mbps, and QUIC dropped to 7.49 Mbps.

These results show that the short path handled moderate TCP loss well, but high loss still caused a major drop in throughput. QUIC was more sensitive to 2% loss in this setup, but both protocols performed poorly at 5% loss.

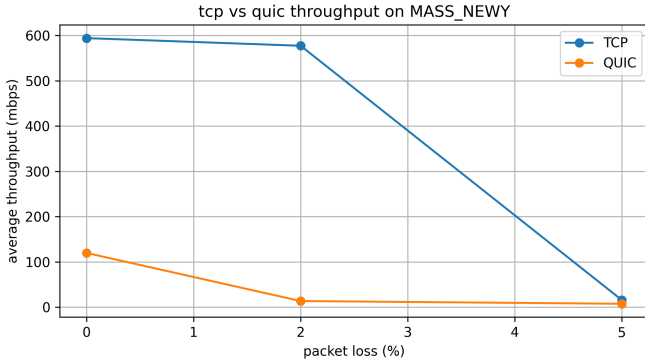


Fig. 1. Average TCP and QUIC throughput under packet loss on the MASS to NEWY path.

C. MASS to UCSD Throughput

Fig. 2 shows the throughput results on the MASS to UCSD path. At 0% loss, TCP reached 71.82 Mbps, while QUIC reached 55.48 Mbps. The baseline values were much lower compared to the MASS to NEWY baseline values. This is expected because the MASS to UCSD was a longer path and had a much higher RTT.

At 2% loss, TCP dropped to 1.82 Mbps, and QUIC dropped to 1.09 Mbps. This means TCP lost 97.46% of its baseline throughput, and QUIC lost 98.04%. At 5% loss, TCP dropped to 0.92 Mbps, and QUIC dropped to 0.63 Mbps.

These results show that the higher-latency path was much more sensitive to packet loss. Even 2% loss caused both protocols to drop to very low throughput. This supports the idea that packet loss and RTT should be studied together.

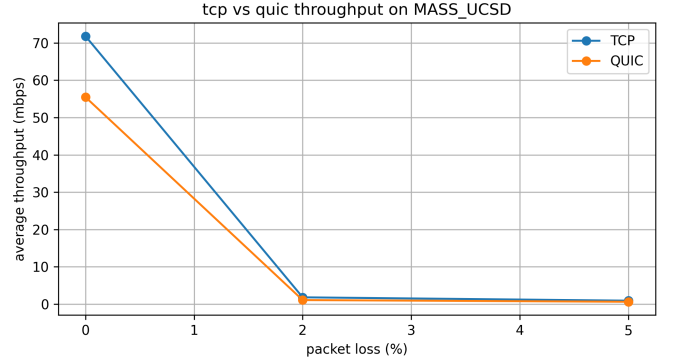


Fig. 2. Average TCP and QUIC throughput under packet loss on the MASS to UCSD path.

Table IV summarizes the average throughput, degradation, and latency results. The degradation summary is useful because raw throughput can be affected by implementation details. In this project, TCP used iperf3, while QUIC used a Python aioquic script. The degradation percentage compares each protocol against its own 0% loss baseline, which makes it easier to see how sensitive each protocol was to packet loss on each path.

D. Throughput Variation Across Trials

Each condition was repeated three times. Table V reports the standard deviation of throughput across the three trials. It helps show whether the average was stable or affected by trial variation.

The MASS to NEWY TCP results had high variation at 0% and 2% loss. This likely happened because the TCP trials were short fixed-size transfers, and the low-latency path could complete the transfer very quickly. Under 5% loss, the same path became much more stable because the throughput dropped sharply in all three trials.

The high standard deviation for TCP on MASS to NEWY should be interpreted carefully. One 0% loss TCP trial reported 172 retransmissions even though no artificial packet loss was applied. Also, one 2% loss TCP trial reached 752.54 Mbps, which was higher than the 0% loss average. These values suggest that the short low-latency TCP tests were affected by normal testbed variation or short-transfer effects. For this reason, the main conclusion is based more on the overall degradation pattern than on any single trial.

The MASS to UCSD results were more consistent under loss. At 2% and 5% loss, the standard deviations were small because both protocols consistently dropped to very low throughput. This supports the main result that the high-latency path was very sensitive to packet loss.

TABLE IV
AVERAGE THROUGHPUT, DEGRADATION, AND LATENCY

Path	Protocol	RTT (ms)	0% Loss Throughput (Mbps)	2% Loss Throughput (Mbps)	5% Loss Throughput (Mbps)	2% Drop	5% Drop
MASS to NEWY	TCP	4.67	594.14	577.28	15.68	2.84%	97.36%
MASS to NEWY	QUIC	4.67	119.72	13.52	7.49	88.71%	93.74%
MASS to UCSD	TCP	71.57	71.82	1.82	0.92	97.46%	98.71%
MASS to UCSD	QUIC	71.57	55.48	1.09	0.63	98.04%	98.87%

TABLE V
THROUGHPUT STANDARD DEVIATION ACROSS THREE TRIALS

Path	Protocol	0% Loss Std. (Mbps)	2% Loss Std. (Mbps)	5% Loss Std. (Mbps)
MASS to NEWY	TCP	140.21	152.21	1.85
MASS to NEWY	QUIC	4.81	0.59	0.12
MASS to UCSD	TCP	3.98	0.03	0.07
MASS to UCSD	QUIC	7.21	0.04	0.02

E. TCP Retransmissions

Table VI reports the average TCP retransmission counts across packet loss levels.

TABLE VI
AVERAGE TCP RETRANSMISSIONS BY PATH AND LOSS LEVEL

Path	0% Loss Retx.	2% Loss Retx.	5% Loss Retx.
MASS to NEWY	57.33	50.67	253.33
MASS to UCSD	0.00	106.00	300.67

On the MASS to NEWY path, TCP retransmissions averaged 253.33 at 5% loss, which matches the expected behavior because TCP uses retransmissions to recover lost data. However, the lower-loss results should be interpreted carefully. One 0% loss TCP trial reported 172 retransmissions even though no artificial packet loss was applied. Because of this unusual baseline trial, the 2% loss average retransmission count was slightly lower than the 0% average on MASS to NEWY. This may have happened because of normal testbed variation, temporary queue behavior, or short-transfer behavior. It does not change the main trend because the controlled 5% loss setting still produced a clear increase in retransmissions and a strong throughput drop.

On the MASS to UCSD path, the retransmission trend was more obvious. At 0% loss, the path had 0 average TCP retransmissions. At 2% loss, it increased to 106.00. At 5% loss, it increased to 300.67. This pattern shows that the artificial loss setting affected TCP recovery behavior.

F. TCP and QUIC Retransmission Visibility

TCP retransmissions were visible because TCP was handled by the Linux kernel. The TCP experiment used iperf3, and

iperf3 reports retransmission counts from the TCP stack. This made TCP retransmissions available in the output.

QUIC retransmissions were not visible in the same way. QUIC runs over UDP, and UDP does not provide retransmission counters. QUIC performs reliability and loss recovery inside the QUIC implementation. In this experiment, that implementation was aioquic. The custom QUIC script printed bytes sent, bytes received, duration, and throughput, but it did not print internal QUIC recovery counters.

For this reason, QUIC retransmission counts were not reported in this experiment. This does not mean that QUIC had zero retransmissions. It means that the experiment did not collect that internal metric. Therefore, QUIC was evaluated through throughput degradation under packet loss.

V. DISCUSSION

The main finding of this experiment is that packet loss becomes much more harmful when path latency is high. This can be explained by the bandwidth-delay product (BDP), which represents the amount of data that can be in flight on a path. When RTT is higher, loss recovery takes longer because feedback and retransmitted data need more time to travel across the network. On MASS to NEWY, TCP could recover from packet loss more quickly because the RTT was under 5 ms. On MASS to UCSD, the RTT was about 15 times larger, so each feedback and recovery cycle took much longer. Since the transfer size was fixed at 8 MB, the high-latency path did not have much time to recover its sending rate before the transfer ended. This explains why 2% loss caused only a 2.84% throughput drop on MASS to NEWY, but a 97.46% drop on MASS to UCSD for TCP, even though both paths used the same loss rate. This shows that packet loss should be considered together with path latency, not by itself.

A similar latency effect appeared in the QUIC results. At 2% loss, QUIC dropped by 88.71% on MASS to NEWY and

by 98.04% on MASS to UCSD. This shows that QUIC was also more sensitive to packet loss on the high-latency path. However, the QUIC results should be interpreted carefully because the experiment used a Python aioquic script rather than a highly optimized production QUIC implementation. Python user-space processing and script-level overhead may have made QUIC more sensitive to loss in this setup. Therefore, these QUIC results describe this specific implementation and experiment, not QUIC in general. This also agrees with prior work showing that the performance of QUIC and TCP depends strongly on network conditions [4], [5].

VI. LIMITATIONS AND FUTURE WORK

This experiment has several limitations. First, the TCP and QUIC tools were not equally optimized. TCP was tested with iperf3, while QUIC was tested with a custom Python program. This may have reduced the QUIC baseline throughput. Future work could test QUIC with a production QUIC or HTTP/3 implementation.

Second, QUIC retransmissions were not directly measured. The script measured throughput, completed bytes, and transfer duration, but it did not expose QUIC recovery counters. Future work could collect QUIC recovery statistics from the library or use packet traces to inspect recovery behavior more directly.

Third, this experiment used an 8 MB transfer size. This helped all tests finish under high loss, but it also means the results reflect fixed-payload transfers rather than long-duration throughput tests. Longer transfers may show different sustained throughput behavior.

At 5% loss on the MASS to UCSD path, individual QUIC transfer trials took approximately 110 seconds, which approached the 160-second client timeout configured in the script. The QUIC throughput at this condition, 0.63 Mbps, may therefore reflect timeout-bounded behavior rather than steady-state congestion control. Future work should increase the timeout or test larger transfers to separate these effects.

Fourth, loss was applied only in the client-to-server direction. This simplified the experiment and kept the data direction consistent. However, real networks may have loss in both directions. Future work could test bidirectional loss or asymmetric loss.

Fifth, this project tested only two FABRIC paths. The two paths were useful because they had very different RTT values, but adding more paths would make the conclusion more general. Future work could add medium-latency paths and more geographic locations.

Future work could also test more loss levels. The current experiment used 0%, 2%, and 5% loss. These points were enough to show the main pattern, but more intermediate values could show where throughput begins to drop sharply. For example, 1% and 3% loss could help identify whether the high-latency path degrades gradually or suddenly.

Finally, a longer experiment could compare congestion control choices. TCP and QUIC can use different congestion control algorithms depending on implementation and system

settings. Controlling this variable more carefully would help separate protocol behavior from congestion control behavior.

VII. CONCLUSION

This project compared TCP and QUIC on two FABRIC paths under controlled packet loss. The main result shows that packet loss has a much stronger effect when path latency is high. At 2% loss, TCP throughput dropped by only 2.84% on the 4.67 ms MASS to NEWY path, but it dropped by 97.46% on the 71.57 ms MASS to UCSD path. QUIC showed the same overall pattern, with much larger degradation on the high-latency path. TCP retransmission counts also confirmed that the injected loss affected transport recovery behavior.

The bandwidth-delay product (BDP) helps explain this result. With higher RTT, each loss event takes longer to recover from, and the sender has less time to rebuild its sending rate during a fixed 8 MB transfer. Because TCP was tested with iperf3 and QUIC was tested with a Python aioquic script, the absolute throughput values should not be treated as a direct protocol comparison. The stronger conclusion is the degradation trend: both protocols were much more sensitive to packet loss on the high-latency path than on the low-latency path. This shows why real wide-area testbed measurements on FABRIC are useful for studying transport performance under different network conditions.

REFERENCES

- [1] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, May 2021.
- [2] M. Bishop, "HTTP/3," RFC 9114, June 2022.
- [3] W. Eddy, "Transmission Control Protocol," RFC 9293, Aug. 2022.
- [4] K. Nepomuceno, I. N. de Oliveira, R. R. Aschoff, D. Bezerra, M. S. Ito, W. Melo, D. Sadok, and G. Szabo, "QUIC and TCP: A performance evaluation," in Proc. IEEE Symposium on Computers and Communications, 2018.
- [5] Y. Yu, M. Xu, and Y. Yang, "When QUIC meets TCP: An experimental study," in Proc. IEEE International Performance Computing and Communications Conference, 2017.
- [6] A. M. Kakhki, S. Jero, D. Choffnes, C. Nita-Rotaru, and A. Mislove, "Taking a long look at QUIC: an approach for rigorous evaluation of rapidly evolving transport protocols," in Proc. Internet Measurement Conference, 2017, pp. 290–303.
- [7] I. Baldin, A. Nikolich, J. Griffioen, I. Monga, K. Wang, T. Lehman, and P. Ruth, "FABRIC: A national scale programmable experimental network infrastructure," IEEE Internet Computing, vol. 23, no. 6, pp. 38–47, Nov. 2019.